Conflict-Based Search for Optimal Multi-Agent Path Finding

A Reproduction and Extension Study

Nimrod Netzer | Elad Damti | Kfir Dahan
Search in Artificial Intelligence — Bar-Ilan University, 2026

Abstract

Multi-Agent Path Finding (MAPF) is the problem of planning collision-free paths for a set of agents on a shared graph, minimizing the total travel cost. Solving MAPF optimally is NP-hard, and naive approaches based on joint-state-space search fail due to exponential state-space growth. In this work, we reproduce key experimental results from Sharon et al. (2015), who introduced Conflict-Based Search (CBS) — a two-level algorithm that separates high-level conflict resolution from low-level single-agent planning. We implement CBS independently in Python and reproduce two central results: the success rate as a function of the number of agents, and a comparison against a Joint-State-Space A* baseline. We further contribute an extension study evaluating how map topology affects CBS performance, comparing open grid maps against warehouse-style maps with narrow corridors and bottlenecks. Our results confirm CBS's advantage over joint search and reveal that structured maps with bottlenecks significantly increase conflict frequency and constraint tree depth, reducing CBS success rates from 64% to 28% at 20 agents.

1. Introduction

Multi-Agent Path Finding (MAPF) arises in many real-world domains including automated warehouses, airport ground traffic management, and railway scheduling [Sharon et al., 2015]. In these settings, a set of agents must navigate from their respective start locations to goal locations on a shared graph without colliding.

The naïve approach of searching the joint configuration space of all agents using A* quickly becomes infeasible. The joint state space grows exponentially with the number of agents: $O(|V|^k)$. For k=10 agents on a graph with |V|=5 vertices, the number of joint states exceeds 9.7 million, making standard search methods impractical for realistic problem sizes.

CBS addresses this by decomposing the problem into a two-level hierarchy. The high level searches a Constraint Tree (CT), branching on conflicts between agent paths. The low level plans optimal paths for individual agents subject to a set of constraints.

In this project, we (1) reproduce two central experimental results from Sharon et al. (2015) — the success rate of CBS vs. Joint-State-Space A* and CT expansion statistics — and (2) contribute an extension study examining how map topology affects CBS performance.

2. Selected Paper Description

2.1 Problem Formulation

The MAPF problem is defined on a graph G=(V,E) with k agents, each with start s_i and goal g_i. At each timestep, an agent moves to an adjacent vertex or waits. A solution is a set of paths with no vertex conflicts (same vertex, same time) or edge conflicts (agents swap positions). The objective is to minimize Sum of Costs: SOC = $\Sigma|\pi_i|$. Optimal MAPF is NP-hard.

2.2 Conflict Types

The paper identifies five conflict types: (1) **Vertex** — same vertex, same time; (2) **Edge** — agents swap positions; (3) **Following** — one follows another; (4) **Cycle** — cyclic dependency; (5) **Swapping** — position exchange. We implement vertex and edge conflicts, sufficient for correctness and optimality.

2.3 Why Joint-State-Space A* Fails

The joint state space grows as $O(|V|^k)$. For k=10 agents on |V|=5 vertices: ~9.7 million states. Our experiments confirm: Joint A* succeeds on only 32% of 6-agent instances and 0% from 8+ agents within a 5-second time limit.

2.4 The CBS Algorithm

Low level — Time-Space A* (TSA*): State = (vertex v, timestep t). A constraint (v,t) forbids agent i from being at v at time t. Heuristic = Manhattan distance (admissible on 4-connected grids). TSA* returns the shortest path for one agent respecting its constraints. Includes future-constraint-aware goal check: agent only stops at goal if no future vertex constraints exist at that location.

High level — Constraint Tree (CT): Binary tree; each node stores constraint sets, paths, and SOC. CBS proceeds: (1) Plan each agent independently — root CT node. (2) Find first conflict. (3) If none — return optimal solution. (4) Branch: child left constrains agent i, child right constrains agent j. (5) Replan constrained agent with TSA*. (6) Add both children to min-heap by SOC. (7) Pop cheapest — repeat from step 2. CBS is complete and optimal.

2.5 CBS Improvements

The paper introduces: Prioritizing Conflicts (cardinal conflicts first), CBS with Heuristics (admissible high-level heuristic), Disjoint Splitting (positive + negative constraints), Conflict Reasoning (larger constraint sets). We implement basic CBS only.

3. Project Description

3.1 Implementation

We implemented CBS independently in Python 3.13 across seven modules: **graph.py** (4-connected Grid); **tsa_star.py** (Time-Space A* with constraint checking and future-constraint-aware goal detection); **conflict.py** (vertex and edge conflict detection); **cbs.py** (CBS high-level constraint tree, min-heap by SOC); **joint_astar.py** (true Joint-State-Space A* baseline — searches all agents simultaneously, matching the paper's intended comparison); **benchmark.py** (map generators, instance generator, experiment runner); **visualize.py**

(matplotlib plots and animations).

3.2 Team Contributions

Nimrod Netzer: tsa_star.py (low-level TSA*, state representation, constraint checking). Wrote Abstract, Sections 1–2.

Elad Damti: conflict.py (conflict detection), cbs.py (CT, CBS loop, CTNode). Wrote Sections 3–4.

Kfir Dahan: joint_astar.py, benchmark.py, visualize.py. Ran all experiments. Wrote Sections 5–6, Reproducibility Statement, AI Disclosure.

All three team members participated in testing, debugging, defense preparation, and the recorded video.

3.3 Key Implementation Choices

Baseline: Joint-State-Space A* — searches joint positions of all agents simultaneously. This matches the paper's intended comparison (unlike independent A* which ignores conflicts).

Goal model: Agents stay at goal indefinitely (paths padded for conflict checking).

Conflict selection: First conflict by timestep, then agent pair index. No prioritization.

Seeds: Instance seed = 42 + n_agents × 1000 + instance_index. Deterministic and reproducible.

4. Experiments

4.1 Experimental Setup

Hardware: 12th Gen Intel Core i7-1255U, 10 cores, 1.7GHz, 16GB RAM, Windows 11 Home. **Software:** Python 3.13, matplotlib 3.11. No external search libraries. **Maps:** 20×20 grid. Open grid: ~10% random obstacles (seed=1). Warehouse grid: alternating shelf rows, single-cell corridors every 4 columns. **Instances:** 25 per agent count, deterministic seeds. **Time limits:** 5s for Joint A* (reproduction baseline); 30s for CBS (reproduction); 15s for CBS (extension). **Differences from paper:** Python vs. C++, 20×20 grid vs. larger benchmarks, shorter time limits.

4.2 Reproduced Results

Table 1 shows CBS vs. Joint-State-Space A* on an open 20×20 grid. Joint A* collapses from 8 agents onward, confirming the exponential state-space explosion. CBS scales gracefully, though it begins timing out beyond 15 agents.

Agents	CBS Success	CBS Time (s)	CT Nodes (mean)	SOC (mean)	Joint A* Success
4	100%	0.0018	1.7	56.8	100%
6	92%	0.0045	3.7	83.8	32%
8	100%	0.0078	5.4	114.3	0%
10	96%	0.0168	12.7	132.6	0%

Agents	CBS Success	CBS Time (s)	CT Nodes (mean)	SOC (mean)	Joint A* Success
12	96%	0.0719	70.6	158.1	0%
15	64%	0.5431	334.3	204.8	0%
18	64%	0.3147	251.3	236.4	0%
20	60%	1.0958	716.3	266.7	0%

Table 1: CBS vs. Joint-State-Space A on open 20×20 grid. 25 instances per agent count. Joint A* time limit: 5s; CBS time limit: 30s. Means over successful instances.*

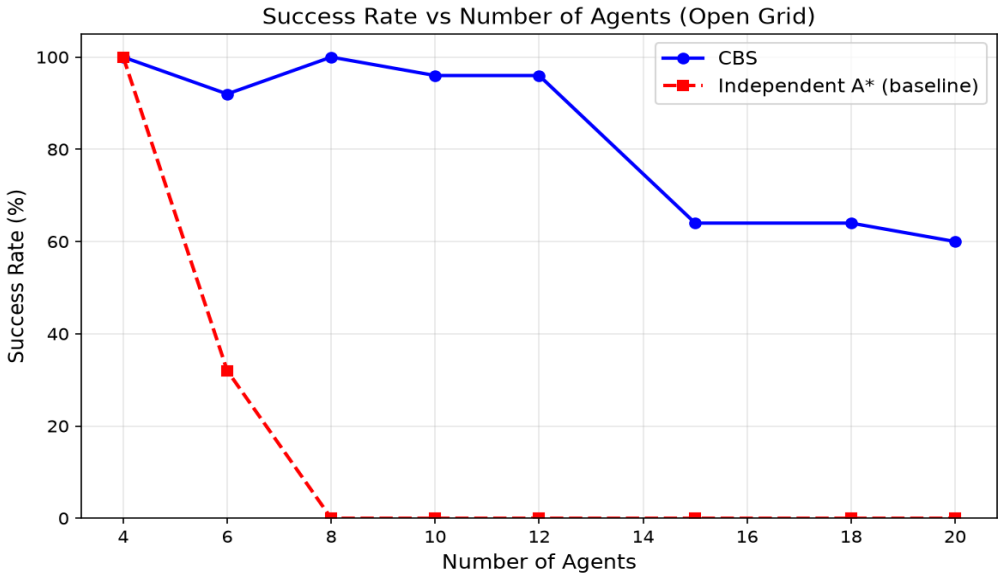

Figure 1: Success rate vs. number of agents — CBS vs. Joint-State-Space A.*

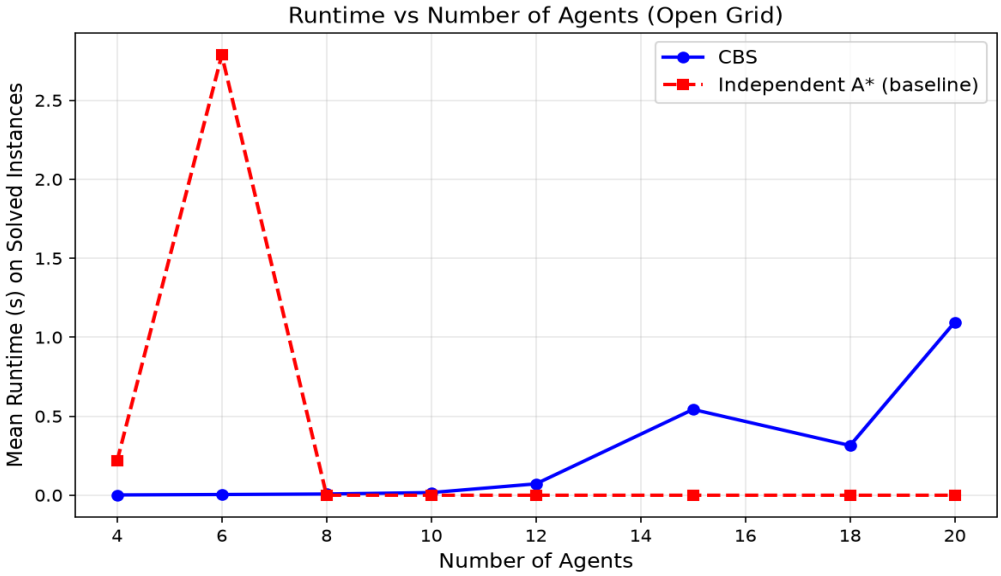

Figure 2: Mean CBS runtime (seconds) on solved instances.

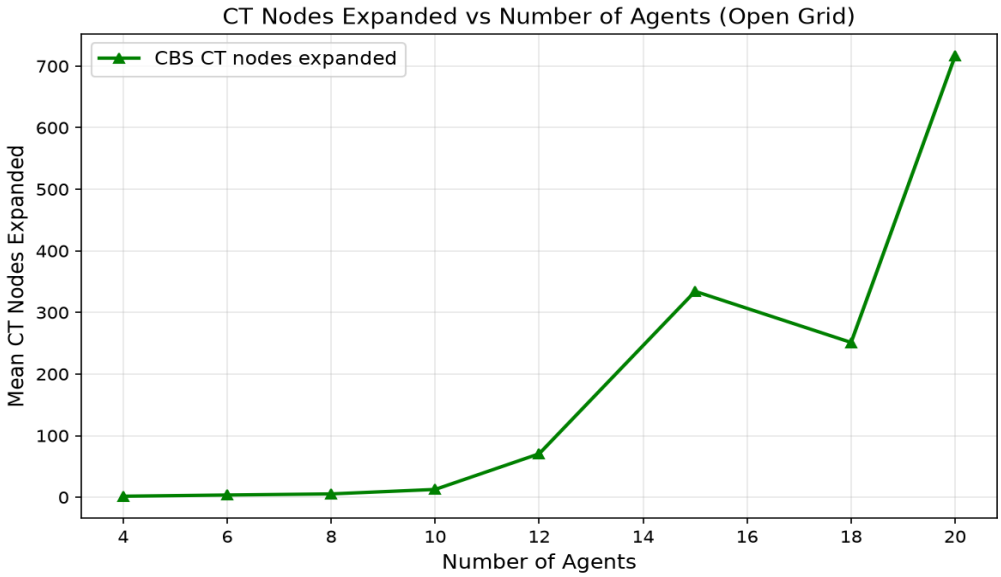

Figure 3: Mean CT nodes expanded vs. number of agents.

A key observation: CT nodes jump from 70.6 at 12 agents to 334.3 at 15 agents — reflecting the exponential worst-case behavior of CBS when many conflicts must be resolved simultaneously, consistent with Sharon et al. (2015).

4.3 Extension Results — Map Topology Study

Research question: Do warehouse-style maps with narrow corridors and bottlenecks create more conflicts and reduce CBS success compared to open grids?

Agents	Open Success	Open CT Nodes	Open Time (s)	Warehouse Success	Warehouse CT Nodes	Warehouse Time (s)
4	100%	1.7	0.0017	100%	3.9	0.0037
6	92%	3.7	0.0044	100%	3.2	0.0031
8	100%	5.4	0.0071	100%	19.4	0.0148
10	96%	12.7	0.0135	100%	125.9	0.2617
12	100%	131.7	0.3525	96%	633.7	0.8381
15	72%	1047.1	1.6304	72%	655.3	0.5128
18	68%	920.1	0.9647	36%	1845.1	1.6920
20	64%	877.4	1.5390	28%	3258.3	3.1362

Table 2: CBS on Open Grid vs. Warehouse Grid. 25 instances per agent count. Time limit: 15s.

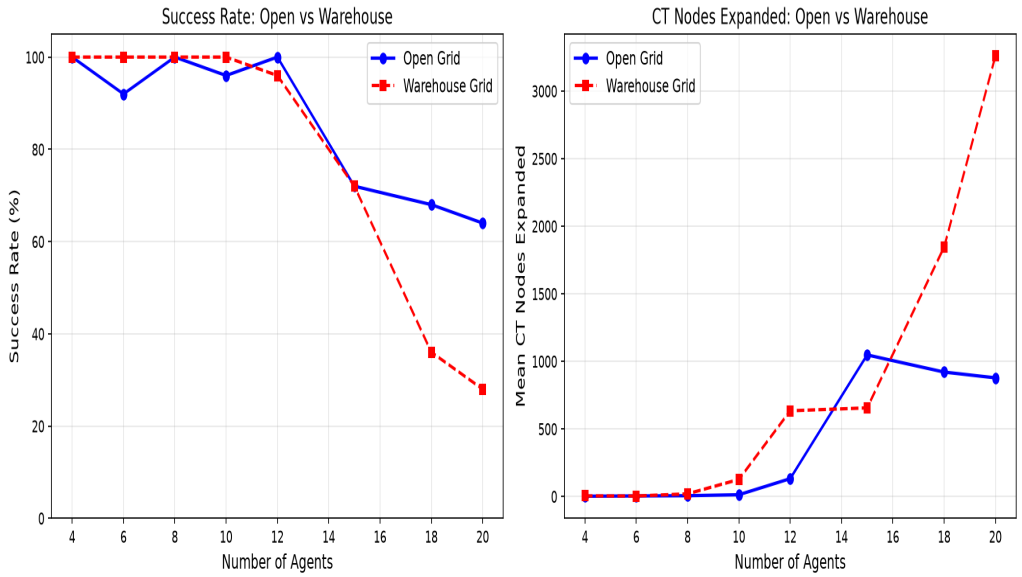

Figure 4: CBS success rate — Open Grid vs. Warehouse Grid.

The warehouse grid dramatically underperforms at high agent counts: 20 agents — Open 64% vs. Warehouse 28%. CT nodes at 20 agents: Open 877 vs. Warehouse 3,258 (3.7×). Bottlenecks force agents into the same corridor cells, multiplying conflicts and CT branches.

5. Discussion

5.1 Agreement with Original Paper

Our results are qualitatively consistent with Sharon et al. (2015): CBS achieves near-perfect success for small agent counts and degrades gracefully; Joint-State-Space A* fails completely from 8 agents; CT expansions grow exponentially beyond ~12 agents. Quantitative differences are expected due to Python vs. C++, smaller grid, and shorter time limits.

5.2 Extension Findings

Warehouse maps increase conflict density due to bottlenecks: agents must funnel through narrow corridors, creating vertex conflicts that force CT branches. The constrained agent takes longer detours, creating new conflicts in adjacent corridors. At 20 agents, CBS expands 3,258 CT nodes on warehouse vs. 877 on open — a 3.7× difference. This confirms that CBS improvements (conflict prioritization, high-level heuristics) would be most beneficial in warehouse-like environments.

5.3 Limitations

Python is ~10–50× slower than C++; 20×20 grids are smaller than paper benchmarks; basic CBS lacks improvements that would extend the success range.

6. Conclusion

We implemented CBS from scratch in Python and reproduced its two central experimental results. CBS finds provably optimal, collision-free solutions while scaling significantly better than Joint-State-Space A*, which fails completely from 8 agents. Our extension study shows that warehouse bottlenecks reduce CBS success from 64% to 28% at 20 agents and increase CT expansions by 3.7×. Future work should apply CBS improvements specifically to warehouse environments where the performance gap is largest.

References

Sharon, G., Stern, R., Felner, A., & Sturtevant, N. R. (2015). Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219, 40–66.

Sturtevant, N. R. (2012). Benchmarks for grid-based pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2), 144–148.

Reproducibility Statement

Results reproduced: (1) Success rate of CBS vs. Joint-State-Space A* as a function of agent count. (2) Runtime and CT node expansion statistics.

Results NOT reproduced: Experiments on Moving AI benchmark maps; comparisons to ICTS or CBS improvement variants.

Implementation: New independent implementation in Python 3.13. Original authors' code was not used.

Changes from original: 20×20 grid (vs. larger benchmarks), 5s/30s time limits, Python instead of C++, randomly generated maps.

Agreement: Qualitative trends match. Absolute numbers differ due to language, grid size, and time limit.

Files: graph.py, tsa_star.py, conflict.py, cbs.py, joint_astar.py, benchmark.py, visualize.py, main.py; results/reproduce_open.csv, extension_open.csv, extension_warehouse.csv; fig1–fig4.png.

How to run:

```
pip install matplotlib

python main.py --mode reproduce --n-instances 25 --time-limit 30 --agent-counts 4
6 8 10 12 15 18 20

python main.py --mode extension --n-instances 25 --time-limit 15 --agent-counts 4
6 8 10 12 15 18 20
```

AI Tools Disclosure

Claude (Anthropic, claude-sonnet-4-6): Used for implementation assistance (code structure, TSA* and CBS implementation, debugging), report writing, algorithm explanations, and

project planning. All code was reviewed, understood, and validated by the team.

The team takes full responsibility for the correctness, originality, and quality of all submitted work.